

1.4 Camera & GPIO Interfaces

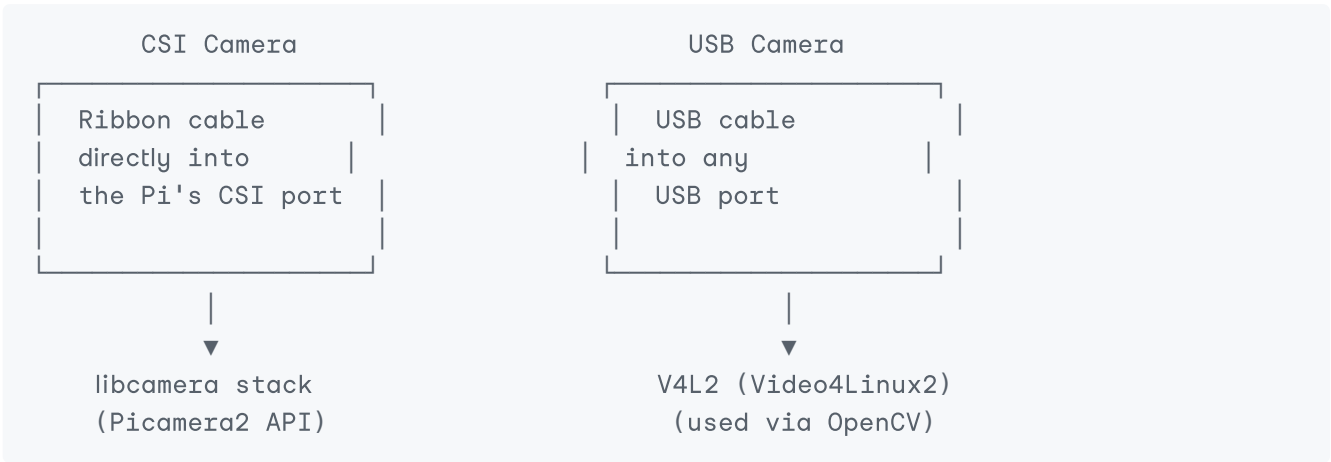
Edge AI Engineering Bootcamp | Analog Data

Why This Matters

This is where the Raspberry Pi stops being just a small Linux computer and starts becoming an **Edge AI device** — something that can see the world through a camera and *act* on it through physical pins (LEDs, relays, buttons). Everything in this module is the bridge between software and the real world.

1. CSI vs USB Cameras

There are two completely different ways to connect a camera to a Raspberry Pi, and they behave very differently in software.



Feature	CSI Camera (e.g., Pi Camera Module 3)	USB Camera (webcam)
Connection	Ribbon cable, dedicated CSI port	Any USB port
Driver stack	libcamera (Bookworm's official camera stack)	V4L2 — the generic Linux video driver framework used by almost any USB camera
Python API	Picamera2	OpenCV (cv2.VideoCapture)
Performance	Usually better — direct hardware pipeline, lower latency	Depends heavily on the camera; USB bandwidth can be a bottleneck
Flexibility	Only works with Pi-compatible camera modules	Works with almost any standard USB webcam

Important change to know: On older Raspberry Pi OS versions (before Bookworm), the camera stack was different (`raspistill` , `raspivid` , or the legacy camera stack). **Bookworm switched entirely to**

`libcamera` as the official, supported way to use CSI cameras. If you find old tutorials using `raspistill`, that command **no longer works** on a fresh Bookworm install — this trips up a lot of students following outdated guides.

Quick check — is your CSI camera detected?

```
libcamera-hello --list-cameras
```

Quick check — is your USB camera detected?

```
v4l2-ctl --list-devices
```

2. Picamera2 Python API — Working with the CSI Camera

Picamera2 is the official Python library for controlling CSI cameras through `libcamera` on Bookworm.

```
# Usually pre-installed on Raspberry Pi OS, but if needed:  
sudo apt install -y python3-picamera2
```

Basic capture example

```
from picamera2 import Picamera2  
import time  
  
picam2 = Picamera2()  
picam2.start()  
time.sleep(2) # give the camera time to adjust exposure/white balance  
  
picam2.capture_file("photo.jpg")  
picam2.close()
```

`capture_array()` — Getting the Image as Raw Data (Not a File)

For AI/vision work, you usually don't want to save a file and re-load it — you want the image **directly in memory** as a numpy array, ready to feed into OpenCV or a model.

```
from picamera2 import Picamera2  
  
picam2 = Picamera2()  
picam2.start()  
  
frame = picam2.capture_array() # returns a numpy array — same format OpenCV/AI  
models expect  
print(frame.shape)           # e.g. (480, 640, 4) → height, width, channels  
  
picam2.close()
```

Why `capture_array()` matters: every AI inference pipeline needs image data as an array of numbers, not a JPEG file on disk. Saving to disk and reading it back is slow — `capture_array()` skips that step entirely and hands you the pixels directly.

Format Selection

Cameras can capture in different pixel formats and resolutions. You configure this *before* starting the camera:

```
from picamera2 import Picamera2

picam2 = Picamera2()

config = picam2.create_preview_configuration(
    main={"size": (640, 480), "format": "RGB888"}
)
picam2.configure(config)
picam2.start()
```

Format	Meaning
RGB888	Standard 3-channel color (Red, Green, Blue) — most common for AI models
YUV420	A format often used for video — smaller size, needs conversion for most AI models
BGR888	Like RGB but channel order swapped — OpenCV's native default

Teaching point: Picking the right format *before* capturing saves you from doing manual color conversion (`cv2.cvtColor`) on every single frame — which matters a lot when you're trying to hit a target FPS.

3. USB Camera — `cv2.VideoCapture()`

USB cameras are accessed through OpenCV directly, using the V4L2 driver underneath.

```
import cv2

cap = cv2.VideoCapture(0)  # 0 = first detected camera device (/dev/video0)

ret, frame = cap.read()    # ret = True/False (did it succeed), frame = the image
if ret:
    cv2.imwrite("photo.jpg", frame)

cap.release()
```

Buffer Flush — A Very Common Bug

USB cameras often keep a small internal buffer of frames. If your program pauses (e.g., waiting on an AI model to finish), by the time you call `cap.read()` again, you might get an **old, stale frame** instead of the current live view.

```
import cv2

cap = cv2.VideoCapture(0)

# Flush the buffer by reading and discarding a few frames first
for _ in range(5):
    cap.read()

ret, frame = cap.read() # NOW this is a fresh, current frame
```

Why this happens: the camera keeps capturing in the background even while your code is busy doing something else (like running inference). Those frames pile up in a buffer. If you don't clear it, you're always looking slightly into the past — which is a real problem for real-time applications like a robot reacting to what it currently sees.

FPS Control

```
import cv2

cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FPS, 30) # request a target FPS
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640) # set resolution width
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480) # set resolution height

actual_fps = cap.get(cv2.CAP_PROP_FPS) # check what the camera actually gave you
print(f"Requested 30 FPS, camera reports: {actual_fps}")
```

Important reality check: setting `CAP_PROP_FPS` is a *request*, not a guarantee — the actual camera hardware may not support the exact value you asked for, and will silently pick its closest supported setting. Always verify with `.get()` after setting.

4. GPIO — Controlling the Physical World

GPIO (General Purpose Input/Output) pins are how the Raspberry Pi talks to the physical world — reading a button press, lighting an LED, triggering a relay.

Raspberry Pi GPIO Header (simplified)

3V3	[●] [●]	5V
GPIO2	[●] [●]	5V
GPIO3	[●] [●]	GND
GPIO4	[●] [●]	GPIO14
GND	[●] [●]	GPIO15
...	(26/40 pins total depending on board)	

There are two common Python libraries for GPIO control:

gpiozero — Simple, Beginner-Friendly

```
from gpiozero import LED, Button
from time import sleep

led = LED(17)          # LED connected to GPIO pin 17
button = Button(2)     # Button connected to GPIO pin 2

while True:
    if button.is_pressed:
        led.on()
    else:
        led.off()
    sleep(0.1)
```

`gpiozero` handles a lot of the low-level details automatically, and reads almost like plain English — great for quickly wiring up a sensor or actuator.

RPi.GPIO — Lower-Level, More Control

```
import RPi.GPIO as GPIO
import time

LED_PIN = 17

GPIO.setmode(GPIO.BCM)          # use Broadcom pin numbering (GPIO17, not physical
pin 11)
GPIO.setup(LED_PIN, GPIO.OUT)   # set pin 17 as an OUTPUT

try:
    while True:
        GPIO.output(LED_PIN, GPIO.HIGH)  # turn LED on
        time.sleep(1)
        GPIO.output(LED_PIN, GPIO.LOW)   # turn LED off
        time.sleep(1)
finally:
    GPIO.cleanup()               # always release the pins when done
```

When to use which:

Use Case	Recommended Library
Simple LED, button, basic relay projects	<code>gpiozero</code>
Precise timing control, PWM details, complex multi-pin logic	<code>RPi.GPIO</code>

Common example — buttons, LEDs, relays:

```
from gpiozero import LED, Button, OutputDevice
from time import sleep

led = LED(17)
```

```

button = Button(2)
relay = OutputDevice(27, active_high=True, initial_value=False)

def on_press():
    led.on()
    relay.on()          # e.g., trigger a fan or pump

def on_release():
    led.off()
    relay.off()

button.when_pressed = on_press
button.when_released = on_release

while True:
    sleep(1)

```

5. I2C, SPI, UART – Talking to Sensors

These are the three main **communication protocols** used to connect sensors and peripherals to the Pi (beyond simple GPIO on/off).

I2C	SPI	UART
2 wires (SDA, SCL) Many devices share the same 2 wires	4 wires (MOSI, MISO, SCLK, CS) Fast, 1 device per CS pin	2 wires (TX, RX) Simple, point-to- point connection

Protocol	Wires Needed	Speed	Typical Use
I2C	2 (SDA, SCL) — shared bus, many devices	Moderate	Temperature sensors, OLED displays, IMUs
SPI	4 (MOSI, MISO, SCLK, CS) — one CS pin per device	Fast	Displays, ADCs, some sensors needing high speed
UART	2 (TX, RX) — direct point-to-point	Moderate	GPS modules, serial debugging, some older sensors

Checking I2C Devices — `i2cdetect -y 1`

```
i2cdetect -y 1
```

This scans the I2C bus (bus 1 on most Pi boards) and shows a grid of addresses. Any address that shows up **means a device is connected and responding** at that address.

Example output:

```
      0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:      -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- 76 -- -- -- -- -- -- -- -- --
```

Here, **76** is a detected device address (commonly a BMP280 temperature/pressure sensor). If the grid is completely empty (**--** everywhere), the sensor either isn't wired correctly, isn't powered, or I2C isn't enabled at all.

Before this will work, remember from Module 1.3: I2C must be enabled first, either via:

```
sudo raspi-config    # Interface Options → I2C → Enable
```

or directly in `/boot/config.txt` :

```
dtoverlay=i2c_arm=on
```

Debugging rule of thumb: if `i2cdetect -y 1` shows nothing, don't assume your sensor is broken — first check wiring, then check power, **then** check that I2C is actually enabled. This single command is your first diagnostic step for almost any sensor problem.

Hands-On Lab: Headless Camera + GPIO Test

Goal: SSH into the Pi headlessly, capture 100 camera frames, measure FPS, and blink an LED every 10th frame.

```
"""
lab_camera_gpio.py
Run this over SSH — no monitor needed.
"""

from picamera2 import Picamera2
from gpiozero import LED
import time

LED_PIN = 17
NUM_FRAMES = 100
BLINK_EVERY = 10
```

```

led = LED(LED_PIN)

picam2 = Picamera2()
picam2.configure(picam2.create_preview_configuration(
    main={"size": (640, 480), "format": "RGB888"}
))
picam2.start()
time.sleep(2) # let exposure/white balance settle

print(f"Capturing {NUM_FRAMES} frames...")
start_time = time.time()

for i in range(1, NUM_FRAMES + 1):
    frame = picam2.capture_array()

    if i % BLINK_EVERY == 0:
        led.on()
        time.sleep(0.05)
        led.off()
        print(f"Frame {i}: blinked LED, shape={frame.shape}")

end_time = time.time()
elapsed = end_time - start_time
fps = NUM_FRAMES / elapsed

print(f"\nCaptured {NUM_FRAMES} frames in {elapsed:.2f} seconds")
print(f"Average FPS: {fps:.2f}")

picam2.close()
led.close()

```

Run it over SSH:

```

ssh yourusername@mypi.local
python3 lab_camera_gpio.py

```

Expected output pattern:

```

Capturing 100 frames...
Frame 10: blinked LED, shape=(480, 640, 3)
Frame 20: blinked LED, shape=(480, 640, 3)
...
Frame 100: blinked LED, shape=(480, 640, 3)

Captured 100 frames in 4.83 seconds
Average FPS: 20.71

```

What this lab actually teaches:

- Working completely headless — no monitor, everything controlled and observed through SSH
- Real camera throughput measurement (your *actual* FPS, not the number on a spec sheet)

- Combining vision (camera) and action (GPIO) in the same loop — the core pattern behind almost every Edge AI project you'll build in this bootcamp
- Reading `frame.shape` to confirm your format/resolution settings are actually being applied

Extension challenge (optional): Modify the script to also print `vcgencmd measure_temp` every 25 frames, so you can see whether sustained camera + GPIO activity raises the Pi's temperature during the test.

Quick Recap

1. **CSI cameras** use `libcamera` / Picamera2; **USB cameras** use V4L2/OpenCV — know which stack you're working with.
 2. `capture_array()` gives you image data directly as a numpy array — no file save/load needed, and picking the right **format** upfront avoids extra conversion work.
 3. USB cameras need **buffer flushing** to avoid stale frames, and **FPS settings are requests**, not guarantees — always verify with `.get()`.
 4. `gpiozero` is simple and readable; `RPi.GPIO` gives lower-level control — both talk to the same physical pins.
 5. **I2C/SPI/UART** are the three main ways sensors connect; `i2cdetect -y 1` is your first diagnostic step when a sensor isn't responding.
-

Self-Check Questions

1. What's the key difference in software stack between a CSI camera and a USB camera?
2. Why is `capture_array()` more useful than saving to a file first, for an AI pipeline?
3. What problem does "buffer flushing" solve on a USB camera, and why does it happen?
4. If `i2cdetect -y 1` shows an empty grid, what are the three things you should check, in order?
5. In the hands-on lab, why do we call `time.sleep(2)` right after `picam2.start()` before capturing any frames?
6. Why is measuring *actual* FPS in your own code more useful than trusting a camera's spec sheet number?